Python OOP Cheat Sheet (Part 1)

Classes, Objects, Instance Attributes, and Constructor Syntax

Senior Technical Consultant
Contact: java.kumar.arun@gmail.com

CLASSES & OBJECTS

A **Class** is a user-defined blueprint or prototype from which real-world entities called **Objects** are instantiated.

Defining a Minimal Class:

```
class Student:
    pass  # Placeholder keyword

# Creating an Object (Instantiation)
s1 = Student()
print(type(s1))  # <class '__main__.Student'>
```

The Naming Convention:

Class names follow *PascalCase* rules (e.g., `TechnicalTrainer`), while object names use lowercase with underscores.

THE INSTANCE CONSTRUCTOR

The `__init__` method automatically triggers whenever a new object is initialized. It assigns specific data parameters onto the state variables.

```
class Trainer:
    def __init__(self, name, topic):
        self.name = name    # Instance variable
        self.topic = topic  # Instance variable

# Initializing objects with unique states
t1 = Trainer("Arun", "Python OOP")
print(t1.name)  # Output: Arun
```

THE MAGIC OF "SELF"

The `self` parameter represents the current instance object currently executing the runtime instructions inside a method block.

It is not a reserved keyword in Python, but it is an absolute mandatory standard convention.

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def show_brand(self):
        # Accessing instance state via self
        print(f"This car is a {self.brand}")

my_car = Car("Nexon")
my_car.show_brand()  # Passes 'my_car' as 'self'
```

INSTANCE METHODS

Instance methods define the behaviors or operational functions that an individual object can carry out.

```
class Account:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        return self.balance

acc = Account(1200000) # Initializing 12 LPA
acc.deposit(50000)
print(acc.balance)     # Output: 1250000
```

CLASS VS INSTANCE SCOPE

Instance Attributes: Bound to a single, specific object. Changes affect no other sibling entities.

Class Attributes: Shared across all initialized instances of that group. Defined directly inside the class body.

```
class Employee:
    company = "Brain Mentors" # Class variable

    def __init__(self, name):
        self.name = name       # Instance variable

e1 = Employee("Jaivik")
e2 = Employee("Maulik")

print(e1.company)  # Brain Mentors
print(e2.company)  # Brain Mentors
print(e1.name)     # Jaivik
```

STRING REPRESENTATION

Override the magic method `__str__` to customize how an object is printed as a string for users.

```
class Book:
    def __init__(self, title):
        self.title = title
    def __str__(self):
        return f"Book Title: {self.title}"

b = Book("Data Analytics Guide")
print(b)  # Out: Book Title: Data Analytics Guide
```

Python OOP Cheat Sheet (Part 2)

The Four Pillars of Object-Oriented Programming Architecture

Senior Technical Consultant
Contact: java.kumar.arun@gmail.com

1. INHERITANCE & SUPER

Allows a child class to absorb attributes and methods from a parent class, promoting code reuse.

```
class Vehicle:
    def __init__(self, make):
        self.make = make
    def move(self):
        print("Vehicle is moving")

class ElectricCar(Vehicle):
    def __init__(self, make, battery_size=70):
        # Call parent constructor via super()
        super().__init__(make)
        self.battery_size = battery_size

tesla = ElectricCar("Tesla", 85)
tesla.move()  # Inherited method execution
```

2. POLYMORPHISM

Polymorphism allows different classes to define methods with the exact same name but uniquely specific local logic execution.

```
class Cat:
    def speak(self): return "Meow"
class Dog:
    def speak(self): return "Woof"

def animal_sound(animal_obj):
    print(animal_obj.speak())

animal_sound(Cat())  # Outputs: Meow
animal_sound(Dog())  # Outputs: Woof
```

3. ENCAPSULATION

Restricts direct variable access to protect data integrity. Prefix underscores block or map properties securely.

- `_variable`: Protected (Internal warning)
- `__variable`: Private (Triggers Name Mangling)

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private field

    def get_balance(self):   # Getter method
        return self.__balance

account = BankAccount(50000)
# print(account.__balance)  # Throws
AttributeError
print(account.get_balance()) # Safe Authorized
Access
```

4. ABSTRACTION

Hides complex implementation details, forcing children to inherit specific structural design schemas via the `abc` standard module.

```
from abc import ABC, abstractmethod

class Appliance(ABC): # Abstract Class
    @abstractmethod
    def set_timer(self):
        pass

class AC(Appliance):
    # Mandatory structural override configuration
    def set_timer(self):
        print("AC Timer set for 2 hours.")

# app = Appliance() # Crucial: Raises TypeError
panasonic_ac = AC()
panasonic_ac.set_timer()
```

QUICK DIAGNOSTIC CHECK

Useful built-in diagnostics helper routines:

```
# Check instance inheritance link
print(isinstance(tesla, Vehicle)) # True

# Check sub-class derivation paths
print(issubclass(ElectricCar, Vehicle)) # True
```